

Bagian 3: Teori & Analisis Algoritma Binary Tree

a. Pohon Ekspresi & Traversal

- **Langkah-demi-langkah `_buildTree()`:** Fungsi membaca *queue* token secara berurutan (*FIFO*).
 1. Membaca `(` pertama: Membuat node dan memanggil rekursi untuk membuat *left child*.
 2. Membaca `(` kedua: Kembali membuat node dan memanggil rekursi untuk *left child*.
 3. Membaca `8`: Menetapkan 8 sebagai *operand* (leaf) lalu *return* (naik).
 4. Membaca `*`: Menetapkan operator `*` ke node *parent*, lalu memanggil rekursi untuk *right child*.
 5. Membaca `5`: Menetapkan 5 sebagai leaf, lalu *return*.
 6. Membaca `)`: Token dibuang/diabaikan, menyelesaikan penyusunan subtree kiri `(8*5)`.
 7. Membaca `+`: Menetapkan root dengan `+`, memanggil rekursi *right child*, lalu mengulang tahap pencabangan yang sama untuk mem-parsing subtree kanan `(9/(7-4))`.
- **Postorder vs Inorder:** Traversal *postorder* menjamin bahwa kedua *operand* anak selalu dikunjungi dan dievaluasi terlebih dahulu sebelum operator akar *parent*-nya (*Left-Right-Visit*). Hal ini secara alamiah menghasilkan format postfix tanpa ambiguitas. Sebaliknya, traversal *inorder* menyisipkan operator di tengah (*Left-Visit-Right*), sehingga memerlukan pencetakan tanda kurung () eksplisit sebelum memanggil sub-traversal dan setelah kembali, guna menghindari hilangnya urutan hierarki presedensi matematis.
- **Kedalaman Rekursi Maksimum:** Fungsi `_buildString()` menelusuri langsung ke bawah mengikuti jalur subtree (*depth-first*). Jika tinggi pohon adalah h , kedalaman maksimum tumpukan eksekusi (*stack*) memori adalah sebanding dengan jumlah level tersebut, bernilai $O(h)$.

b. Struktur Heap & Pemetaan Array

- **Validitas Rumus Heap:** Rumus pemetaan logis indeks $\text{parent} = (i-1)/2$, $\text{left} = 2*i+1$, dan $\text{right} = 2*i+2$ bertumpu mutlak pada properti *complete binary tree* di mana node level terbawah terisi penuh dari kiri ke kanan tanpa adanya "lubang" (indeks kosong) di tengah *array*. Apabila pohon tidak *complete*, elemen yang cacat logis tersebut akan merusak keselarasan pergeseran indeks statis di dalam *array* memori sekuensial.
- **Pemulihan dengan `sift-down()`:** Saat akar (nilai terbesar) diekstraksi, heap mencopot node daun terakhir (kanan bawah) dan meletakkannya kembali di posisi

akar agar panjang list mengecil stabil $O(1)$ secara ruang. Nilai akar baru ini cenderung kecil sehingga merusak urutan properti akar. `sift-down()` bertugas membandingkan node ini dengan kedua anaknya secara berulang, dan menukar posisinya (*swap*) dengan anak yang bernilai lebih besar, hingga turun menemukan posisi perhentian validnya di bagian bawah heap.

- **Perbandingan Maksimum:** Setiap simpul `sift-down()` melakukan maksimum 2 perbandingan (satu untuk mencari anak terbesar, satu untuk membandingkannya dengan induk). Karena tinggi pohon seimbang adalah $\lceil \log_2 n \rceil$, jumlah maksimal perbandingannya dibatasi sekitar $2 \lceil \log_2 n \rceil$ kali.

c. Heapsort In-Place vs Simple

- **Perbandingan Kompleksitas Ruang & Memori:** * *Simple Heapsort* memakan ruang ganda ($O(n)$) untuk alokasi penampungan urutan elemen baru hasil ekstraksi di luar *heap array* asli.
 - *In-Place Heapsort* beroperasi dengan mempartisi struktur `theSeq` yang sama menjadi bagian "zona heap aktif" (kiri) dan "zona data terurut" (kanan array), dengan nol alokasi ruang array luar/murni $O(1)$. Penggunaan memori ini lebih menguntungkan untuk *cache locality* dan mereduksi habis-habisan risiko interupsi *stack-overflow* RAM.
- **Retensi Kompleksitas Waktu:** Walau memakai array yang sama dengan teknik iterasi swap, waktu *in-place heapsort* konstan di tingkat $\Theta(n \log n)$. Ini terjadi karena operasi `sift-down` logaritmik dijalankan berulang per iterasi n , dan sifat operasi pertukaran variabel belaka (*swap*) adalah konstan $O(1)$ yang tidak akan menaikkan hierarki asimtotiknya melampaui $n \log n$.

d. Batas Teoretis & Decision Tree

- **Status Teoretis Heap:** Heapsort tergolong *Comparison Sort*, ia sepenuhnya mengandalkan evaluasi `left >= largest` antar pasangan node di `sift-down()`. Waktu optimal Heapsort $O(n \log n)$ tepat mendarat persis di lantai fundamental perbandingan sorting $\Omega(n \log n)$, sehingga tidak ada kontradiksi sama sekali.
- **Decision Tree Morse Code:** Saat merekonstruksi urutan sinyal yang diterjemahkan, pohon Morse menguji karakter *dot* (*left*) atau *dash* (*right*) menuruni level. Jika suatu saat pembacaan menavigasi paksa menuju *null link* (*None*), hal ini secara deterministik mendeteksi sebuah kebuntuan rute logis, membuktikan urutan *token* bersifat *invalid* tanpa perlu mencocokkan *library array*.
- **Keunggulan Rekursi:** Pohon biner adalah representasi geometrik rekursif secara *built-in*, yang di mana setiap *sub-node* melahirkan pohonnya sendiri.

Implementasi dengan rekursi mendelegasikan beban mengingat posisi cabang (*backtracking*) kepada OS dan *call stack*, jauh lebih rapi dibanding manipulasi iteratif yang akan memaksa programmer menyiapkan arsitektur *custom stack* secara manual.